

C++ + Python + Ollama (Jarvis System Guide)

Grundidee

C++ dient als Hauptprogramm (CLI, Steuerung), während Python die KI-Logik übernimmt. C++ ruft Python auf und bekommt die Antwort zurück.

Python Script (ai.py)

```
import sys

frage = sys.argv[1]

print(f"Python hat bekommen: {frage}")
```

C++ Code (main.cpp)

```
#include <iostream>
#include <cstdio>
#include <memory>
#include <array>
#include <string>

std::string runPython(const std::string& input) {
    std::string command = "python3 ai.py \"" + input + "\"";
    std::array<char, 256> buffer{};
    std::string result;

    FILE* pipe = popen(command.c_str(), "r");
    if (!pipe) {
        return "Fehler beim Starten von Python.";
    }

    while (fgets(buffer.data(), buffer.size(), pipe) != nullptr) {
        result += buffer.data();
    }

    pclose(pipe);
    return result;
}

int main() {
    std::string antwort = runPython("Hallo Jarvis");
    std::cout << antwort << std::endl;
    return 0;
}
```

Was passiert hier?

- C++ startet Python mit popen() - Python verarbeitet die Anfrage - Python gibt Text zurück - C++ liest die Antwort und zeigt sie an

Jarvis Architektur

Nutzer → C++ (Router) → Python (KI) → Ollama → Antwort → C++ → Ausgabe

Wichtige Hinweise

- Sonderzeichen im Input können Probleme machen - Für größere Projekte besser Dateien oder Pipes nutzen - Python eignet sich ideal für KI-Logik

Nächste Schritte

1. Ollama integrieren 2. Router (FAST / SMART) in C++ bauen 3. Tools hinzufügen (Dateien, Rechner, etc.)